

07 JS Arrays

JavaScript Arrays: Complete Tutorial & Lab Guide

Introduction to Arrays

What is an Array? An array is a special variable that can hold multiple values at once. Think of it as a container with numbered compartments (starting from 0) where you can store different items.

Real-world analogy: Imagine a row of lockers. Each locker has a number (index) and can store something inside (value).

```
Array: fruits
Index:   0         1         2         3
Value: "apple"  "banana" "orange" "mango"
```

Lab Setup

Step 1: Create Your Project Folder

1. Create a new folder called `js-arrays-lab` on your desktop
2. Open VS Code
3. Click **File** → **Open Folder** and select `js-arrays-lab`

Step 2: Create Your Files

1. Create a new file: `index.html`
2. Create a new file: `script.js`

Step 3: Set Up HTML File

Copy this into `index.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>JavaScript Arrays Lab</title>
</head>
<body>
  <h1>🚀 JavaScript Arrays Lab</h1>
  <div id="output">
    <p>Open the console (F12) to see the results!</p>
  </div>
  <script src="script.js"></script>
</body>
</html>
```

Step 4: Start Live Server

1. Right-click on `index.html` in VS Code
2. Select **"Open with Live Server"**
3. Your browser will open automatically
4. Press **F12** to open Developer Tools (Console tab)

You're ready to code! 🎉

Creating Arrays

Method 1: Array Literal (Most Common)

Open `script.js` and add:

```
// Creating an array of fruits
let fruits = ["apple", "banana", "orange"];
console.log("Fruits:", fruits);

// Creating an array of numbers
let numbers = [10, 20, 30, 40, 50];
console.log("Numbers:", numbers);

// Creating an array with mixed data types
let mixedArray = ["John", 25, true, null];
console.log("Mixed:", mixedArray);

// Creating an empty array
let emptyArray = [];
console.log("Empty:", emptyArray);
```

Save and check your browser console!

Method 2: Using the Array Constructor

Add this to your `script.js`:

```
// Using Array constructor
let colors = new Array("red", "green", "blue");
console.log("Colors:", colors);

// Creating an array with a specific length
let emptySlots = new Array(5);
console.log("Empty slots:", emptySlots);
console.log("Length:", emptySlots.length);
```

💡 **Tip:** Array literal `[]` is preferred because it's shorter and clearer.

Accessing Elements

Arrays use **zero-based indexing** (counting starts at 0).

Add to `script.js`:

```
console.log("≡ ACCESSING ELEMENTS ≡");

let animals = ["cat", "dog", "elephant", "lion", "tiger"];

// Access individual elements
console.log("First animal:", animals[0]);    // cat
console.log("Second animal:", animals[1]);   // dog
console.log("Third animal:", animals[2]);    // elephant

// Access last element
console.log("Last animal:", animals[animals.length - 1]); // tiger

// Trying to access non-existent index
console.log("Index 10:", animals[10]);      // undefined

// Get array length
console.log("Total animals:", animals.length);
```

Visual representation:

```
animals = ["cat", "dog", "elephant", "lion", "tiger"]
index:    0      1      2      3      4
```



Modifying Arrays

Add to `script.js`:

```
console.log("≡ MODIFYING ARRAYS ≡");

let shoppingCart = ["milk", "bread", "eggs"];
console.log("Original cart:", shoppingCart);

// Change an existing element
shoppingCart[1] = "butter";
console.log("After changing bread to butter:", shoppingCart);

// Add element at the end
shoppingCart[3] = "cheese";
console.log("After adding cheese:", shoppingCart);

// Add element beyond current length (creates empty slots)
shoppingCart[6] = "cookies";
console.log("After adding at index 6:", shoppingCart);
console.log("Length:", shoppingCart.length);
```

Array Methods

Adding and Removing Elements

Add to `script.js`:

```
console.log("≡ ADDING & REMOVING ELEMENTS ≡");

let todos = ["wake up", "brush teeth"];
console.log("Initial todos:", todos);

// push() - Add to end
todos.push("have breakfast");
console.log("After push:", todos);

// unshift() - Add to beginning
todos.unshift("open eyes");
console.log("After unshift:", todos);

// pop() - Remove from end
let removed = todos.pop();
console.log("Removed item:", removed);
console.log("After pop:", todos);

// shift() - Remove from beginning
let firstItem = todos.shift();
console.log("Removed first:", firstItem);
console.log("After shift:", todos);
```

Quick Reference:

- `push()` → adds to end
- `pop()` → removes from end
- `unshift()` → adds to beginning
- `shift()` → removes from beginning

Finding Elements

Add to `script.js`:

```
console.log("≡ FINDING ELEMENTS ≡");

let languages = ["JavaScript", "Python", "Java", "C++", "Python"];

// indexOf() - Find first occurrence
let position = languages.indexOf("Python");
console.log("Python is at index:", position);

// lastIndexOf() - Find last occurrence
let lastPos = languages.lastIndexOf("Python");
console.log("Last Python at index:", lastPos);

// includes() - Check if element exists
let hasJava = languages.includes("Java");
console.log("Has Java?", hasJava);

let hasRuby = languages.includes("Ruby");
console.log("Has Ruby?", hasRuby);

// find() - Find first element matching condition
let numbers = [5, 12, 8, 130, 44];
let found = numbers.find(num => num > 10);
console.log("First number > 10:", found);

// findIndex() - Find index of first match
let foundIndex = numbers.findIndex(num => num > 10);
console.log("Index of first number > 10:", foundIndex);
```

Transforming Arrays

Add to `script.js`:

```
console.log("≡≡≡ TRANSFORMING ARRAYS ≡≡≡");

// slice() - Extract portion (doesn't modify original)
let fruits = ["apple", "banana", "orange", "mango", "grape"];
let citrus = fruits.slice(2, 4); // from index 2 to 4 (excluding 4)
console.log("Original fruits:", fruits);
console.log("Sliced (2 to 4):", citrus);

// splice() - Add/remove elements (modifies original)
let colors = ["red", "green", "blue", "yellow"];
console.log("Before splice:", colors);

// Remove 2 elements starting at index 1
let removed = colors.splice(1, 2);
console.log("Removed:", removed);
console.log("After removing:", colors);

// Add elements at index 1
colors.splice(1, 0, "purple", "pink");
console.log("After adding:", colors);

// Replace elements
colors.splice(2, 1, "orange");
console.log("After replacing:", colors);
```

Combining and Splitting Arrays

Add to `script.js`:


```
console.log("≡ COMBINING & SPLITTING ≡");

// concat() - Combine arrays
let array1 = [1, 2, 3];
let array2 = [4, 5, 6];
let combined = array1.concat(array2);
console.log("Combined:", combined);

// join() - Convert array to string
let words = ["Hello", "World", "from", "JavaScript"];
let sentence = words.join(" ");
console.log("Joined with space:", sentence);

let csv = words.join(",");
console.log("Joined with comma:", csv);

// split() - Convert string to array (String method)
let text = "apple,banana,orange";
let fruitsArray = text.split(",");
console.log("Split string:", fruitsArray);

// reverse() - Reverse array (modifies original)
let numbers = [1, 2, 3, 4, 5];
console.log("Before reverse:", numbers);
numbers.reverse();
console.log("After reverse:", numbers);

// sort() - Sort array (modifies original)
let names = ["Charlie", "Alice", "Bob"];
names.sort();
console.log("Sorted names:", names);

// Sorting numbers (needs compare function)
let nums = [40, 100, 1, 5, 25];
nums.sort((a, b) => a - b); // ascending
console.log("Sorted numbers:", nums);
```

Iterating Through Arrays

Method 1: For Loop

Add to `script.js`:

```
console.log("=== ITERATION METHODS ===");

let grades = [85, 92, 78, 95, 88];

// Traditional for loop
console.log("Using for loop:");
for (let i = 0; i < grades.length; i++) {
  console.log(`Grade ${i + 1}: ${grades[i]}`);
}
```

Method 2: For...of Loop

Add to `script.js`:

```
// For...of loop (modern, clean)
console.log("\nUsing for...of loop:");
for (let grade of grades) {
  console.log("Grade:", grade);
}
```

Method 3: forEach Method

Add to `script.js`:

```
// forEach() method
console.log("\nUsing forEach:");
grades.forEach(function(grade, index) {
  console.log(`Index ${index}: ${grade}`);
});

// forEach with arrow function
console.log("\nUsing forEach with arrow function:");
grades.forEach((grade, index) => {
  console.log(`Position ${index + 1}: ${grade} points`);
});
```

Method 4: Map Method

Add to `script.js`:

```
// map() - Transform each element and create new array
console.log("\nUsing map:");
let prices = [10, 20, 30, 40];
let discountedPrices = prices.map(price => price * 0.9);
console.log("Original prices:", prices);
console.log("After 10% discount:", discountedPrices);

// Double each number
let numbers = [1, 2, 3, 4, 5];
let doubled = numbers.map(num => num * 2);
console.log("Original:", numbers);
console.log("Doubled:", doubled);
```

Method 5: Filter Method

Add to `script.js`:

```
// filter() - Create new array with elements that pass test
console.log("\nUsing filter:");
let ages = [15, 22, 18, 30, 16, 25];
let adults = ages.filter(age => age >= 18);
console.log("All ages:", ages);
console.log("Adults only:", adults);

// Filter even numbers
let allNumbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
let evenNumbers = allNumbers.filter(num => num % 2 === 0);
console.log("Even numbers:", evenNumbers);
```

Method 6: Reduce Method

Add to `script.js`:

```
// reduce() - Reduce array to single value
console.log("\nUsing reduce:");
let cartPrices = [29.99, 15.50, 42.00, 8.99];

// Calculate total
let total = cartPrices.reduce((sum, price) => sum + price, 0);
console.log("Cart items:", cartPrices);
console.log("Total:", total.toFixed(2));

// Find maximum number
let scores = [88, 95, 72, 91, 85];
let maxScore = scores.reduce((max, score) =>
  score > max ? score : max
);
console.log("Scores:", scores);
console.log("Highest score:", maxScore);
```



Advanced Concepts

Multi-dimensional Arrays

Add to `script.js`:

```
console.log("≡ MULTI-DIMENSIONAL ARRAYS ≡");

// 2D Array (Array of arrays)
let matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];

console.log("Matrix:", matrix);
console.log("Element at [0][0]:", matrix[0][0]); // 1
console.log("Element at [1][2]:", matrix[1][2]); // 6
console.log("Element at [2][1]:", matrix[2][1]); // 8

// Practical example: Student grades
let studentGrades = [
  ["Alice", 85, 92, 78],
  ["Bob", 90, 88, 95],
  ["Charlie", 75, 80, 85]
];

console.log("\nStudent Grades:");
studentGrades.forEach(student => {
  let name = student[0];
  let avg = (student[1] + student[2] + student[3]) / 3;
  console.log(`${name}: ${avg.toFixed(1)}`);
});
```

Array Destructuring

Add to `script.js`:

```
console.log("=== ARRAY DESTRUCTURING ===");

// Basic destructuring
let colors = ["red", "green", "blue"];
let [first, second, third] = colors;
console.log("First:", first);
console.log("Second:", second);
console.log("Third:", third);

// Skip elements
let [primary, , tertiary] = colors;
console.log("Primary:", primary);
console.log("Tertiary:", tertiary);

// Rest operator
let numbers = [1, 2, 3, 4, 5];
let [one, two, ... rest] = numbers;
console.log("First two:", one, two);
console.log("Rest:", rest);

// Swapping variables
let a = 1;
let b = 2;
console.log("Before swap: a =", a, ", b =", b);
[a, b] = [b, a];
console.log("After swap: a =", a, ", b =", b);
```

Spread Operator

Add to `script.js`:

```
console.log("=== SPREAD OPERATOR ===");

// Copy array
let original = [1, 2, 3];
let copy = [ ... original];
console.log("Original:", original);
console.log("Copy:", copy);

// Combine arrays
let arr1 = [1, 2, 3];
let arr2 = [4, 5, 6];
let combined = [ ... arr1, ... arr2];
console.log("Combined:", combined);

// Add elements while copying
let base = [2, 3, 4];
let extended = [1, ... base, 5, 6];
console.log("Extended:", extended);

// Find max/min using spread
let values = [15, 8, 42, 23, 16];
let maximum = Math.max( ... values);
let minimum = Math.min( ... values);
console.log("Values:", values);
console.log("Max:", maximum);
console.log("Min:", minimum);
```

Checking if Something is an Array

Add to `script.js`:

```
console.log("=== CHECKING ARRAYS ===");

let realArray = [1, 2, 3];
let notArray = "Hello";
let alsoNotArray = { name: "John" };

console.log("Is [1,2,3] an array?", Array.isArray(realArray));
console.log("Is 'Hello' an array?", Array.isArray(notArray));
console.log("Is {name:'John'} an array?", Array.isArray(alsoNotArray));
```

Array.from() and Array.of()

Add to `script.js`:

```
console.log("=== ARRAY.FROM & ARRAY.OF ===");

// Array.from() - Create array from array-like or iterable
let text = "Hello";
let charArray = Array.from(text);
console.log("String to array:", charArray);

// Create array from range
let range = Array.from({ length: 5 }, (_, i) => i + 1);
console.log("Range 1-5:", range);

// Array.of() - Create array from arguments
let numbers = Array.of(1, 2, 3, 4, 5);
console.log("Array.of:", numbers);

// Difference from Array constructor
let withConstructor = new Array(5); // Creates 5 empty slots
let withOf = Array.of(5);           // Creates array with one element: 5
console.log("new Array(5):", withConstructor);
console.log("Array.of(5):", withOf);
```

JS

Chaining Array Methods

Add to `script.js`:


```
console.log("=== METHOD CHAINING ===");

let products = [
  { name: "Laptop", price: 999, category: "electronics" },
  { name: "Phone", price: 699, category: "electronics" },
  { name: "Shirt", price: 29, category: "clothing" },
  { name: "Shoes", price: 89, category: "clothing" },
  { name: "Watch", price: 199, category: "accessories" }
];

// Complex operation: Filter electronics, get prices, calculate total
let electronicsTotal = products
  .filter(item => item.category === "electronics")
  .map(item => item.price)
  .reduce((sum, price) => sum + price, 0);

console.log("Electronics total:", electronicsTotal);

// Get names of items under $100
let affordableItems = products
  .filter(item => item.price < 100)
  .map(item => item.name);

console.log("Affordable items:", affordableItems);
```



Practice Exercises

Create a new section in your `script.js`:

```
console.log("=== PRACTICE EXERCISES ===");

// Exercise 1: Sum of array elements
console.log("\nExercise 1: Sum of array");
let nums = [10, 20, 30, 40, 50];
let sum = nums.reduce((total, num) => total + num, 0);
console.log("Numbers:", nums);
console.log("Sum:", sum);

// Exercise 2: Find longest word
console.log("\nExercise 2: Longest word");
let words = ["cat", "elephant", "dog", "hippopotamus", "bird"];
let longest = words.reduce((longest, word) =>
  word.length > longest.length ? word : longest
);
console.log("Words:", words);
console.log("Longest:", longest);

// Exercise 3: Remove duplicates
console.log("\nExercise 3: Remove duplicates");
let withDuplicates = [1, 2, 2, 3, 4, 4, 5, 1];
let unique = [ ... new Set(withDuplicates)];
console.log("Original:", withDuplicates);
console.log("Unique:", unique);

// Exercise 4: Count occurrences
console.log("\nExercise 4: Count occurrences");
let fruits = ["apple", "banana", "apple", "orange", "banana", "apple"];
let count = {};
fruits.forEach(fruit => {
  count[fruit] = (count[fruit] || 0) + 1;
});
console.log("Fruits:", fruits);
console.log("Count:", count);

// Exercise 5: Flatten nested array
console.log("\nExercise 5: Flatten array");
let nested = [1, [2, 3], [4, [5, 6]]];
let flattened = nested.flat(2);
console.log("Nested:", nested);
console.log("Flattened:", flattened);
```

Summary

Key Takeaways:

1. **Arrays store multiple values** in a single variable using numbered indices
2. **Zero-based indexing** means the first element is at index 0
3. **Common methods** you'll use daily:
 - `push()`, `pop()` for end operations
 - `shift()`, `unshift()` for start operations
 - `map()` to transform elements
 - `filter()` to select elements
 - `reduce()` to calculate single values
 - `forEach()` to iterate
4. **Arrays are mutable** - most methods modify the original array
5. **Method chaining** lets you combine operations elegantly

Next Steps:

- Experiment with different array methods
 - Try combining methods in creative ways
 - Build small projects like todo lists or shopping carts
 - Practice with the exercises above
-

Quick Reference Card

JS

```
// Creating
let arr = [1, 2, 3];

// Accessing
arr[0]           // First element
arr[arr.length - 1] // Last element

// Adding/Removing
arr.push(4)      // Add to end
arr.pop()        // Remove from end
arr.unshift(0)   // Add to start
arr.shift()      // Remove from start

// Finding
arr.indexOf(2)   // Find index
arr.includes(3)  // Check existence
arr.find(x => x > 2) // Find element

// Transforming
arr.map(x => x * 2) // Transform each
arr.filter(x => x > 1) // Select matching
arr.reduce((a,b) => a+b) // Combine to one

// Iterating
arr.forEach(x => console.log(x))
for (let item of arr) { }

// Others
arr.slice(1, 3)   // Extract portion
arr.splice(1, 2)  // Remove/add elements
arr.concat([4, 5]) // Combine arrays
arr.join(', ')    // Convert to string
arr.reverse()     // Reverse order
arr.sort()        // Sort elements
```

Happy Coding! 🚀